SkillPulse — A Complete Beginner’s Guide to CI/CD with GitHub Actions

Build a real production pipeline. Put it on your resume. Talk about it in interviews.

TrainWithShubham GitHub Actions & Kubernetes Masterclass

May 2026

Foreword — read this first

This guide assumes nothing. If you have never:

- pushed code to GitHub,
- written a Dockerfile,
- SSHed into a Linux server,
- or heard the words “CI/CD,”

you can still finish this guide and end up with a real, working pipeline running on a real cloud server, deploying real Docker containers, with real secrets, automatically, on every git push.

That is a portfolio project. You can:

- Add it to your resume.
- Send the GitHub link to recruiters.
- Talk through it in interviews.
- Reuse the same setup as the skeleton for your own apps.

This document walks through exactly what was built, why each piece exists, and — at the end — how to talk about it to a hiring manager.

Part 1 — The Big Picture

What we built

A small skill-tracking web application called **SkillPulse**, with a fully automated pipeline that:

1. Detects every push to the `main` branch on GitHub.
2. Builds two Docker container images (frontend and backend).
3. Tags each image with the exact commit hash *and* with `latest`.
4. Pushes both to Docker Hub.
5. SSHes into an Amazon EC2 server.
6. Pulls the new images.
7. Restarts only the containers whose images actually changed.
8. Cleans up old, unused images.

Total YAML to make all of this happen: about fifty lines.

The full developer experience becomes:

> *Edit a file. Commit. Push. Ninety seconds later, your change is live on the public internet.*

Once you have built this once, you understand 80% of what every modern engineering team does between writing code and shipping it.

Why this matters

Three reasons.

1. Hiring signal. Every DevOps, SRE, Cloud Engineer, or Platform Engineer job posting asks for some combination of: "experience with CI/CD pipelines," "Docker," "GitHub Actions / Jenkins / GitLab CI," "cloud deployment." This project hits all of them — visibly, on GitHub, with commit history that tells the story.

2. Foundation. Kubernetes, GitOps, service meshes, observability — they all sit on top of CI/CD. You cannot meaningfully learn the next layer until this one is intuitive.

3. It's real. Most tutorials end at "now the test passes locally." This one ends at "now I push code and it deploys to a server I can curl from anywhere in the world." The difference matters.

Part 2 — Foundations

The minimum theory you need before touching any code. Skim or skip if you already know this.

What is DevOps?

For decades, the people who wrote software (developers) and the people who ran software (operations) were two different teams with two different goals.

- Developers wanted to ship features. Their bonus depended on it.
- Operations wanted nothing to break. Their pager depended on it.

These goals are in tension. The fastest way for ops to be stable was to slow developers down — long change-approval boards, rare releases, freeze windows. The fastest way for developers to ship was to throw code over the wall and let ops figure out how to run it. Both teams resented each other. Customers waited months for fixes.

DevOps is the cultural and technical answer to that tension. The core idea:

> **The same team owns the change all the way to production, and tooling makes that safe.**

DevOps is not a job title. It's a way of working. When it is working you can recognize it because:

- Deploys are boring. Friday afternoon? Sure.
- Rollbacks are cheap — a stuck deploy gets reverted in 30 seconds.
- Feedback is fast — a broken commit fails in minutes, not "after QA next sprint."
- Ownership is clear — the person who wrote the code watches it ship.

The way you get there is by automating the path from a developer's laptop to production. That automation is called a **pipeline**.

What is CI/CD?

Two ideas in one acronym.

Continuous Integration (CI). Every change, from every developer, gets built and tested automatically the moment it lands. You catch breakage in minutes. Merge conflicts shrink because nobody's branch lives for two weeks before integrating.

Continuous Delivery / Deployment (CD). Every change that passes CI is automatically packaged and shipped — to staging, or all the way to production. There is no "deploy day." Every commit is a candidate release.

The non-obvious thing — the thing every CI/CD beginner misses — is this:

> **CI doesn't just test your code. It produces an artifact.**

That artifact (in our case, a Docker image) is what production runs. If the artifact is built consistently in CI, it is the same in dev, staging, and prod. "Works on my machine" stops being a possible failure mode, because nothing is being built on anyone's machine — it's being built once, in CI, and used everywhere.

What is a container?

A container is a way of packaging an application together with everything it needs to run — its operating system libraries, its language runtime, its config — into a single immutable image that can be shipped and run anywhere Docker (or any compatible runtime) is installed.

The mental model: a container is a tiny shipping crate. The contents are sealed. The crate runs the same way on your laptop, in CI, and on production servers. There is no "but my version of Node is different" anymore.

Two terms you'll mix up at first:

- **Image** — the packaged thing on disk. Static. Like a class definition.
- **Container** — a running instance of an image. Dynamic. Like an object.

You build an image, push it to a registry (Docker Hub), then on the server you pull it and run it as a container.

What is GitHub Actions?

A pipeline needs a **runner** — a machine that watches your repo, executes your build/test/deploy steps, and reports back.

Historically that meant standing up a Jenkins server (slow, heavy, requires its own maintenance), paying for CircleCI or Travis, or rolling your own. All of those still exist. None are the lowest-friction option in 2026.

GitHub Actions wins on three things:

1. **It lives where the code lives.** Your workflows are YAML files inside the repo, in `.github/workflows/`. They evolve with the code. They are reviewed in the same pull requests. They survive every clone.
2. **It is free for public repos and generous for private ones.** A complete CI/CD pipeline costs zero rupees to start.
3. **The Marketplace is enormous.** Need to SSH into a server? `appleboy/ssh-action`. Need to log into Docker Hub? `docker/login-action`. You compose pre-built blocks instead of writing bash from scratch.

The trade-off is GitHub vendor lock-in. For most teams, that's a fair price for the integration.

What is EC2?

Amazon EC2 (Elastic Compute Cloud) is, in plain English, *renting a Linux computer in Amazon's data center, by the hour*.

You log into the AWS console, click "launch instance," pick an Ubuntu image, pick a size (t3.micro is enough to learn — it costs roughly Rs 1 per hour, less if you are inside the AWS free tier), and ninety seconds later you have a Linux box on the public internet with an IP you can SSH into.

For this project, EC2 is the production server. The CI workflow builds images on GitHub's runners. The CD workflow SSHes into your EC2 and tells it to pull and run those images.

In real production you would use Kubernetes (or ECS, or Fargate, or Cloud Run) instead of a single EC2 — that is the second half of the masterclass. But every production engineer started by learning to deploy onto a single VM, and you should too.

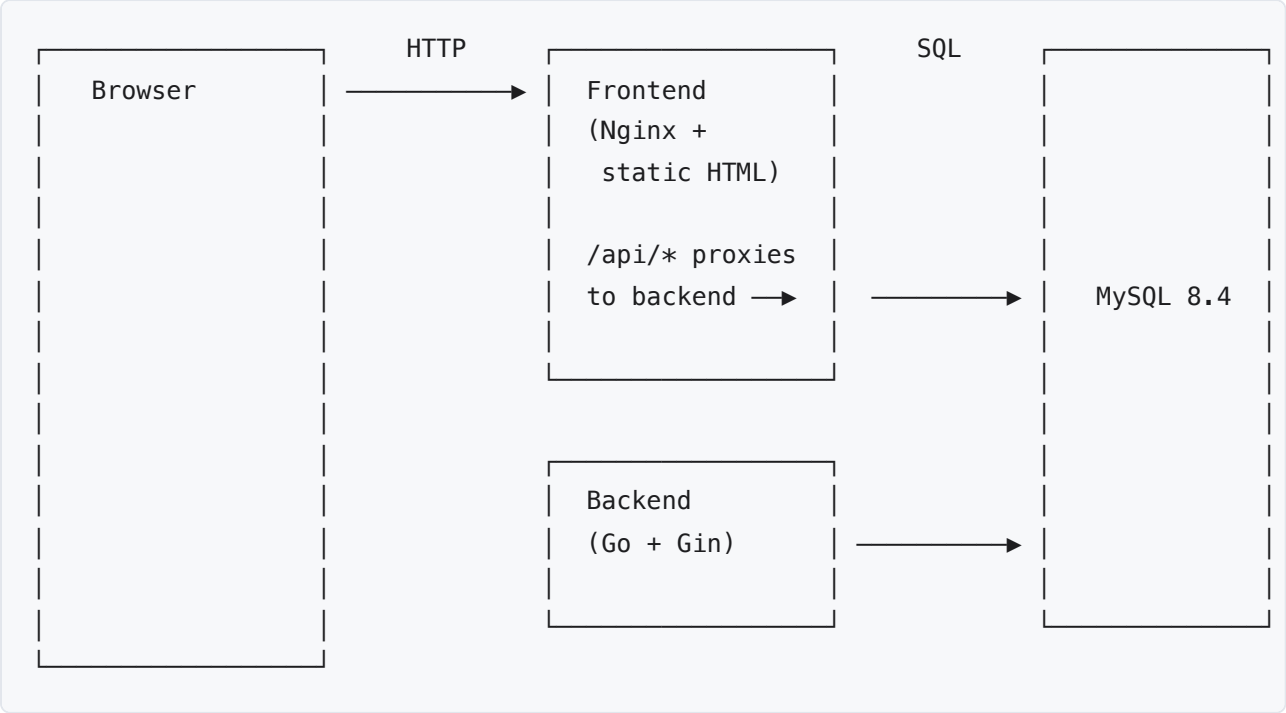
Part 3 — The Application

The application itself is intentionally small, because the application is not the point — the pipeline around the application is the point. But you should understand what is being deployed.

SkillPulse — what it does

SkillPulse is a “track your learning” tool. You add a skill (e.g. “Docker,” “Kubernetes”), set a target number of hours, and log study sessions against it. There’s a dashboard showing total skills, total hours, and your top skill.

It has three tiers — frontend, backend, database — exactly like most real web applications.



The three tiers

Frontend — `frontend/`. Plain HTML, CSS, and vanilla JavaScript. No build step, no React, no webpack. The image is `nginx:alpine` with the static files baked in. Nginx also acts as a reverse proxy: requests to `/api/*` are forwarded to the backend container.

Backend — `backend/`. A Go service using the Gin web framework. It exposes a small REST API. It uses the standard library `database/sql` package — no ORM. SQL queries live inline in the handlers, which is fine for a project this size and much easier to read than ORM-style code if you are new to Go.

Database — `mysql/`. MySQL 8.4 with a single SQL file (`init.sql`) that creates two tables (`skills`, `learning_logs`) and inserts some seed data. The Docker compose file mounts this SQL

file into a special directory — `/docker-entrypoint-initdb.d/` — that the official MySQL image automatically runs on first boot. The database "initializes itself."

The API

```
GET    /api/skills              list all skills + total hours per skill
POST   /api/skills              create a skill
GET    /api/skills/:id          one skill + its logs
DELETE /api/skills/:id          delete a skill (logs cascade)
POST   /api/skills/:id/log      log a study session
GET    /api/dashboard           summary counters
GET    /health                  database ping (for healthchecks)
```

If you have never seen REST routes before: each line is one URL pattern + one HTTP verb that the backend will respond to. The frontend's JavaScript calls these.

Part 4 — The Pipeline

Now the centerpiece. We have two YAML files in `.github/workflows/`:

- `ci.yml` — builds and pushes Docker images.
- `cd.yml` — deploys to the EC2.

Together they're about 50 lines of YAML.

CI — `ci.yml` line by line

```yaml
name: CI

on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-buildx-action@v4

      - uses: docker/login-action@v4
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push backend
        uses: docker/build-push-action@v7
        with:
          context: ./backend
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-backend:${{ github.sha
        }}
            ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-backend:latest

      - name: Build and push frontend
        uses: docker/build-push-action@v7
        with:
          context: ./frontend
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-frontend:${{ github.sha
        }}
            ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-frontend:latest
```

`name: CI`. A label that shows up in the GitHub Actions UI.

`on: push: branches: [main]`. Triggers the workflow whenever someone pushes to `main`. Why limit to main? Because feature branches shouldn't ship to production. CI on every branch could be a separate workflow.

`runs-on: ubuntu-latest`. Each workflow run starts on a freshly-provisioned Ubuntu virtual machine. No state carried over from previous runs. This is why "works on my laptop" failures get caught in CI — the runner has none of your laptop's quirks.

`actions/checkout@v4`. Clones your repo into the runner. Without this step, the runner has no code.

`docker/setup-buildx-action@v4`. Sets up Buildx, Docker's modern builder. Lets us use multi-stage builds and produce smaller images.

`docker/login-action@v4`. Authenticates the runner against Docker Hub using the secrets `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`. Without authentication, you cannot push images to your account.

The two `build-push-action` steps. Each builds an image (one for backend, one for frontend) and pushes it. Notice the `tags:` block has two tags per image:

- `:${{ github.sha }}` — the full commit hash. Immutable. *This is your rollback handle.* If the deploy after commit `abc1234` breaks production, you can deploy `:def5678` and you know exactly what code you're running.
- `:latest` — a moving pointer to the most recently pushed build. *This is what production pulls.* The CD workflow uses `:latest`, never `:sha`.

This dual-tag pattern is one of the most important habits in container CI. If you only tag `:latest`, you lose history. If you only tag `:sha`, deployment becomes complicated. Tag both. Always.

CD — `cd.yml` line by line

```yaml
name: CD

on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
    branches: [main]

jobs:
  deploy:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            if [ ! -d ~/skillpulse ]; then
              git clone ${{ github.server_url }}/${{ github.repository }}.git
        ~/skillpulse
            fi
            cd ~/skillpulse
            git pull origin main
            if [ ! -f .env ]; then
              echo "ERROR: ~/skillpulse/.env is missing. Create it once with
        DOCKERHUB_USERNAME and DB credentials."
              exit 1
            fi
            docker compose pull
            docker compose up -d
            docker image prune -f
```

`on: workflow_run`. This is the trigger that chains CD to CI. The CD workflow waits for the CI workflow to complete, then runs.

`if: conclusion == 'success'`. *This is critical.* Without this `if`, CD would run even when CI failed — meaning a broken commit would still try to deploy. With it, a failed CI cleanly skips CD. Try it on purpose: introduce a typo in the Dockerfile. CI will fail. CD will appear in the Actions tab with a “skipped” status, not “failed.” That’s the gate working.

`appleboy/ssh-action@v1`. A pre-built action that SSHes into a server and runs a script. The three secrets it needs:

- `EC2_HOST` — the public IP of the server.
- `EC2_USER` — usually `ubuntu` for Ubuntu AMIs.
- `EC2_SSH_KEY` — the *contents* of your private key (`.pem`) file, pasted in. Not a path.

Now the deploy script itself:

`if [ ! -d ~/skillpulse ]; then git clone ... fi`. Idempotent. The first time the workflow runs on a fresh server, it clones the repo. Every subsequent run, the directory exists, so the clone is skipped. No script that fails on the second run.

`git pull origin main`. Brings the deploy-side checkout up to date with what just got pushed. (We need this because docker-compose.yml lives in the repo and references things like the MySQL init script — so the EC2 needs the latest copy of those.)

The `.env` existence check. If the file is missing, deploy refuses to continue *with a useful error message*. Without this check, `docker compose` would fail with cryptic "$DOCKERHUB_USERNAME is empty" messages and you'd waste an hour debugging. Fail fast, fail clearly.

`docker compose pull`. Downloads the freshly-pushed `:latest` images from Docker Hub onto the EC2.

`docker compose up -d`. Starts (or restarts) the containers in detached mode. *Critically: `up -d` only restarts the services whose images actually changed.* If you only edited frontend HTML, the backend container does not bounce. The database container does not bounce. Zero unnecessary downtime.

You will sometimes see tutorials that do `docker compose down && docker compose up -d` instead. **That is wrong** — `down` brings everything down, including services that didn't change. We deliberately removed that pattern.

`docker image prune -f`. Deletes dangling images (the old `:sha`-tagged ones that are no longer referenced). Without this, after weeks of deploys, the EC2 disk fills up with old image layers and you wake up to a server that won't boot.

Secrets — what they are and why they matter

A secret in GitHub Actions is an environment variable you store in repo settings (Settings → Secrets and variables → Actions). Workflows reference them as `${{ secrets.NAME }}`. Workflow logs *automatically mask* secret values — even if a step accidentally `echo`'s a secret, the value shows as `***` in the logs.

Our five secrets:

Secret	What it is	Where you get it
`DOCKERHUB_USERNAME`	Your Docker Hub account name	hub.docker.com — your username
`DOCKERHUB_TOKEN`	Personal Access Token, read+write	Docker Hub → Account Settings → Personal access tokens
`EC2_HOST`	Public IP or DNS of the EC2	AWS Console → EC2 → Instances
`EC2_USER`	Linux user on the EC2	`ubuntu` for Ubuntu AMIs
`EC2_SSH_KEY`	Private key file *contents*	The `.pem` file you downloaded when you launched the EC2

The reason secrets exist: anything that authorizes you (passwords, tokens, private keys) must never appear in your repo's source code. If you commit them by accident, anyone who clones your repo can impersonate you. Secrets keep them in GitHub's encrypted store, only injected into workflow runs.

Part 5 — Build It Yourself, Step by Step

This is the part where you actually do the thing.

Prerequisites

You will need (free) accounts on:

1. **GitHub** — github.com.
2. **Docker Hub** — hub.docker.com.
3. **AWS** — aws.amazon.com (you'll need a credit/debit card, but if you're inside the 12-month free tier, an EC2 t3.micro is free).

You will need installed locally:

- **Git** — `git --version` should work.
- **Docker Desktop** (Mac/Windows) or Docker Engine (Linux) — `docker --version` should work.
- A **terminal** — Mac Terminal, Linux terminal, or Windows WSL.

Step 1 — Fork the repo

Go to `github.com/LondheShubham153/github-actions-kubernetes-masterclass` and click "Fork." This creates `github.com/<your-username>/github-actions-kubernetes-masterclass`.

Then clone *your fork* locally:

```
git clone git@github.com:<your-username>/github-actions-kubernetes-
        masterclass.git
cd github-actions-kubernetes-masterclass
```

Step 2 — Run it locally first

You should always run a project locally before automating its deployment. If it doesn't work locally, no pipeline can save you.

```
cp .env.example .env
# Edit .env — for local-only, DOCKERHUB_USERNAME can be anything
docker compose up -d --build
```

Open `http://localhost`. You should see SkillPulse with five seed skills.

To stop:

```
docker compose down
```

To wipe the database too:

```
docker compose down -v
```

Step 3 — Set up Docker Hub

Go to hub.docker.com, sign up, then:

1. Click your username (top right) → Account Settings → Personal access tokens.
2. Generate a new token with **Read & Write** scope. Name it `github-actions-skillpulse`.
3. **Copy the token immediately.** It is shown only once.

You now have:

- A username (e.g. `your-handle`).
- A token (starts with `dckr_pat_...`).

Step 4 — Launch an EC2 instance

In the AWS Console:

1. Region: pick the one closest to you (e.g. `ap-south-1` for Mumbai, `us-east-1` for N. Virginia).
2. EC2 → Launch instance.
3. Name: `skillpulse-demo`.
4. AMI: **Ubuntu Server 24.04 LTS**.
5. Instance type: **t3.micro** (free tier eligible) or **t2.micro**.
6. Key pair: create a new one, name it `skillpulse-key`, type `.pem`. **Download it.** This is the only chance to download the private key.
7. Security group — allow inbound:
   - Port `22` (SSH) from "My IP."
   - Port `80` (HTTP) from `0.0.0.0/0` (anywhere).
8. Launch.

Wait until the instance is in "Running" state. Note the **public IPv4 address**.

Move and protect the key file:

```
mv ~/Downloads/skillpulse-key.pem ~/.ssh/
chmod 600 ~/.ssh/skillpulse-key.pem
```

SSH in and install Docker:

```
ssh -i ~/.ssh/skillpulse-key.pem ubuntu@<EC2-PUBLIC-IP>

# Inside the EC2 now:
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
exit
```

SSH back in (so the new group membership takes effect) and verify:

```
ssh -i ~/.ssh/skillpulse-key.pem ubuntu@<EC2-PUBLIC-IP>
docker ps      # should print an empty table, no permission errors
```

Create the deploy directory and the `.env` file:

```
mkdir -p ~/skillpulse
cd ~/skillpulse
cat > .env <<'EOF'
MYSQL_ROOT_PASSWORD=rootpassword123
DB_HOST=db
DB_PORT=3306
DB_USER=skillpulse
DB_PASSWORD=skillpulse123
DB_NAME=skillpulse
DOCKERHUB_USERNAME=<your-docker-hub-username>
EOF
exit
```

Step 5 — Add secrets to GitHub

In your forked repo on GitHub:

1. Settings → Secrets and variables → Actions → New repository secret.
2. Add five secrets:

Name	Value
`DOCKERHUB_USERNAME`	your Docker Hub username
`DOCKERHUB_TOKEN`	the token from Step 3
`EC2_HOST`	the EC2 public IP from Step 4
`EC2_USER`	`ubuntu`
`EC2_SSH_KEY`	open the `.pem` file in a text editor and paste the *entire contents* including the `-----BEGIN/END-----` lines

Step 6 — Push and watch the pipeline run

Make any small edit — change a string in `frontend/index.html`. Then:

```
git add frontend/index.html
git commit -m "first deploy"
git push origin main
```

In your repo on GitHub, click the **Actions** tab. You should see:

1. A **CI** workflow running. Click into it and watch the steps tick green: checkout, buildx setup, docker login, build+push backend, build+push frontend. Should take ~60 seconds.
2. About 10 seconds after CI completes, a **CD** workflow appears and runs. Click into it and watch the SSH step execute the deploy script.

Then visit `http://<EC2-PUBLIC-IP>` — your change is live.

Step 7 — Make a real change, watch it ship

```
# Change the page title, for example
sed -i.bak 's|<title>.*</title>|<title>My SkillPulse</title>|'
        frontend/index.html
git add frontend/index.html
git commit -m "rename page title"
git push origin main
```

Watch the Actions tab. ~90 seconds later, refresh the EC2 URL — new title.

This loop — **edit → commit → push → live** — is the entire point of CI/CD. Internalize how short it is.

Step 8 — Break things on purpose

The best way to learn what good failure modes look like is to cause them.

Break 1: invalid Dockerfile. Add `RUN this-command-does-not-exist` to `backend/Dockerfile`. Push.

- CI fails at the build step.
- CD does *not* run. (You'll see it appear with status "skipped.")
- That's the success-gate working.
- Revert your change and push again — both workflows go green.

Break 2: bad Docker Hub token. In repo settings, change `DOCKERHUB_TOKEN` to nonsense. Push any commit.

- CI fails at the docker login step.
- The error message in logs will say "unauthorized: personal access token is expired" or similar.
- Now you know what an expired credential looks like. Restore the real token.

Break 3: missing `.env` on the server. SSH into the EC2, `mv ~/skillpulse/.env ~/.env.bak`. Push any commit.

- CI passes.
- CD fails with the explicit error: `ERROR: ~/skillpulse/.env is missing.`
- This is the kind of fail-fast error message worth its weight in gold. Restore the file.

Part 6 — Engineering Decisions

These are the design choices that turn a "tutorial" pipeline into a "real" pipeline. If you can explain *why* each one is the way it is, you understand pipelines at a level above most beginners.

Why tag images with both `:sha` and `:latest`?

`:latest` alone has no history. If today's `:latest` breaks production, you have no way to redeploy yesterday's version short of rebuilding it from a checkout of yesterday's commit (slow, error-prone). With `:sha`, every image is permanently addressable. *Rolling back becomes editing one tag in the deploy.*

`:sha` alone is too inconvenient for the deploy script — you'd have to inject the SHA into compose. `:latest` keeps the deploy script simple while `:sha` keeps history.

Why does CD use `workflow_run` and not run inside `ci.yml`?

Two reasons.

1. **Separation of concerns.** Build and deploy are different steps with different secrets, different environments, different blast radii. Two files force you to think about them separately.
2. **Reusability.** If you later add a "manual deploy" trigger (deploy a specific SHA on demand), it's a simple `workflow_dispatch` addition to `cd.yml` — no entanglement with CI.

Why the `conclusion == 'success'` guard?

Default `workflow_run` fires on *completion*, regardless of pass/fail. Without the guard, a failed CI would still trigger a deploy attempt — and depending on how your script is written, you might deploy stale images or partially-built ones. The guard makes the gate semantic: deploy only on green.

Why is the SSH script idempotent (`if [ ! -d ~/skillpulse ]`)?

The same script must work for the first deploy (fresh server, no checkout) and the thousandth deploy (existing checkout). Without the `if`, the first run succeeds (cloning), and every subsequent run fails ("destination path already exists"). With it, both work. Idempotence is one of the cardinal virtues of automation.

Why not `docker compose down && docker compose up -d`?

`down` stops *all* services. If you only changed the frontend image, `down` still stops the backend and the database. The database stop+start can take 30+ seconds. Connections drop. For services with persistent state, this is genuinely harmful.

`docker compose up -d` *alone* is smarter: it diffs running containers against the desired state and only restarts what changed. Faster, less downtime, less risk.

Why `docker image prune -f` after every deploy?

Each deploy pulls a new image. The old image (still tagged with its `:sha`, but no longer being used by any running container) sits on disk. After 100 deploys, you have 100 frontend images and 100 backend images on the EC2 — multiple gigabytes. The disk fills, the EC2 stops responding.

`prune -f` removes only *dangling* images (no tags, not used by any container). It is safe. Run it on every deploy.

Why a `.env` existence check?

Without it, `docker compose pull` happily reads empty values for `DOCKERHUB_USERNAME` and tries to pull `/skillpulse-backend:latest` (note the leading slash — that's malformed). The error message is unhelpful. The check turns a 30-minute debug session into a 1-second clear error.

This is a general principle: **fail fast at the point of clearest understanding.** The deploy script knows exactly what it needs. Any precondition should be checked at the start, not deep inside a tool's stack trace.

Part 7 — Putting This on Your Resume

This is what most tutorials don't teach. You built a real thing — now you need to talk about it.

Resume bullets

Pick 3–4 of these (don't list all). Adapt to your actual depth.

> **CI/CD Pipeline — SkillPulse (personal project)**
>
> - Designed and built a full CI/CD pipeline using **GitHub Actions** that automatically builds, tags, pushes, and deploys a multi-service web application to AWS EC2 on every commit to `main`, reducing deploy-time from manual minutes to a fully unattended ~90-second cycle.
> - Containerized a three-tier application (Go backend, Nginx frontend, MySQL database) using **multi-stage Docker builds**, producing minimal production images and pushing them to **Docker Hub** with both immutable SHA tags (for rollback) and a moving `:latest` tag (for deployment).
> - Implemented a `workflow_run`-chained deploy job gated on CI success, an idempotent SSH-driven release script with explicit pre-flight checks, and post-deploy image cleanup, eliminating accidental deploys of broken builds and preventing disk exhaustion on the deploy host.
> - Authored production-quality engineering rationale: dual-tag image strategy, fail-fast `.env` validation, no-op-on-no-change container restarts, secure secret handling via GitHub Actions encrypted secrets.

The wording matters. Words like *designed*, *implemented*, *eliminating*, *authored* signal ownership. Words like *helped*, *worked on*, *learned* signal participation. Always claim the higher one truthfully.

Sample STAR-format interview answer

> **Question:** "Tell me about a project where you set up CI/CD."
>
> **Situation:** I wanted to learn modern DevOps end to end, so I built a small skill-tracker app with a real production pipeline — three services (Go API, Nginx frontend, MySQL), deployed on AWS EC2.
>
> **Task:** I needed every commit to `main` to automatically test, build, push images to a registry, and deploy to the EC2 — without me touching anything.
>
> **Action:** I split the pipeline into two GitHub Actions workflows. The CI workflow builds both Docker images, tags them with the commit SHA *and* `:latest`, and pushes to Docker Hub. The CD workflow uses GitHub's `workflow_run` trigger gated on CI success, SSHes into EC2, idempotently clones the repo, validates the `.env` exists, and runs `docker compose pull` and `up -d`. I added `image prune -f` to keep the disk clean over time.
>
> **Result:** Push-to-prod time dropped to ~90 seconds. Failed builds correctly skip deploys instead of running broken code. Rollback is a one-line change to the image tag. The pipeline has been used for [N] iterations of the app without manual intervention.

This answer is ~150 seconds spoken. Practice it. Time yourself.

Interview questions you can now answer

For each, write your own answer. Do not memorize ours — your answer in your words will sound right; a memorized answer will sound wrong.

1. *What is CI/CD, in your own words?*
2. *Why use containers? Why Docker specifically?*
3. *What's the difference between an image and a container?*
4. *Why would you tag an image with both* `:sha` *and* `:latest`*?*
5. *How do you trigger one GitHub Actions workflow from another?*
6. *Why would* `docker compose down && docker compose up -d` *be worse than* `docker compose up -d` *alone?*
7. *Where do you store secrets so they don't end up in your repo?*
8. *What does "idempotent" mean and why does it matter for deploy scripts?*
9. *How would you roll back a bad deploy with this pipeline?*
10. *What would you do differently if this app went into real production?*

The last one matters most. Honest answer: *"I would add automated tests, use a real secret manager instead of GitHub secrets, deploy to multiple instances behind a load balancer, replace the SSH-based deploy with Kubernetes manifests, and add observability — Prometheus for metrics, structured logs going to a log aggregator. The pipeline I built is a learning skeleton, not a production endpoint."*

A candidate who *knows what they don't know* is more impressive than one who oversells.

What to demo in a screen-share interview

If asked "show me your project," do this in order, in under 5 minutes:

1. Show the **GitHub repo** — point at `.github/workflows/`. "Here are my two workflows. CI builds and pushes images. CD deploys."
2. Open `ci.yml`. **Read aloud the dual-tag block.** Explain the rollback story.
3. Open `cd.yml`. **Point at the `if: conclusion == 'success'` line.** Explain why it matters.
4. Show the **Actions tab** — the green checkmarks, the durations.
5. Open the **live URL** in a browser. "This is the EC2."
6. Make one tiny visible change in code, push, switch to the Actions tab. While CI is still running, talk through what is happening. When CD finishes, refresh the live URL — change visible.

That last step is the showstopper. *Edit → push → live, in under two minutes, on a real server, on a real call.* Most interviewers haven't seen a candidate do this. You will be remembered.

Part 8 — Going Further

You have just built the foundation. Here is what to learn next, in roughly this order.

1. Add real tests. The CI right now only verifies "the image builds." Add a `go test` step before the docker build. Make a test fail on purpose; watch CI block the deploy. Now CI is testing *and* building.

2. Multi-environment deploys. Add a `staging` branch and a `staging.yml` workflow that deploys to a separate EC2 (or just a different `~/skillpulse-staging` directory on the same EC2). Promote staging → prod by merging.

3. Real secret management. Move `DB_PASSWORD` out of `.env` and into AWS Secrets Manager or HashiCorp Vault. Have the deploy script fetch it at runtime. This is a common interview question and a real production pattern.

4. Replace the SSH deploy with Kubernetes. This is the next half of the masterclass. Same app, same CI workflow producing the same images — but instead of `appleboy/ssh-action`, the CD workflow runs `kubectl apply` against an EKS / minikube / kind cluster. Suddenly you have:

- Zero-downtime rolling deploys.
- Multi-replica horizontal scaling.
- Self-healing (a crashed pod restarts automatically).
- Liveness and readiness probes.
- Secrets via Kubernetes Secrets or external operators.

5. GitOps. Replace "CD pushes to the cluster" with "the cluster pulls from a manifests repo." Tools: **Argo CD**, **Flux**. The cluster becomes the source of truth's *consumer*, not its *target*. Auditable, reversible, declarative.

6. Observability. Add Prometheus to scrape backend metrics, Grafana to visualize, Loki for logs, and configure alerts. Now you have a full SRE-shaped setup, not just a CI/CD setup.

7. Infrastructure as code. Right now you provisioned the EC2 by clicking in the AWS console. Replace that with a Terraform module. Now your *infrastructure itself* is in version control and applied via CI.

Each of those is a multi-week learning project of its own. None are theoretical — they are all things you can build, push to GitHub, and put on your resume.

Appendix A — Glossary

Action — a reusable building block in GitHub Actions, like `actions/checkout`. Combines into workflows.

Artifact — something a build produces. In our case, a Docker image.

CI (Continuous Integration) — automatically testing every change as it lands in version control.

CD (Continuous Delivery / Deployment) — automatically packaging and shipping changes that pass CI.

Container — a running instance of an image. Isolated process with its own filesystem, network, and resources.

Container Registry — a service that stores container images. Docker Hub, GitHub Container Registry, ECR, GCR.

Docker Compose — a tool for running multiple containers together with shared networking, defined by a YAML file.

Dockerfile — the recipe for building an image.

EC2 — Amazon Elastic Compute Cloud. Rentable Linux/Windows servers in AWS.

Idempotent — an operation that can be repeated and produces the same result. Critical for deploy scripts.

Image — a static, immutable snapshot of an application and its dependencies, ready to run as a container.

Multi-stage build — a Dockerfile pattern where one stage compiles the app and a smaller stage runs it. Reduces final image size.

Pipeline — the chain of automation from code commit to running production.

Rollback — redeploying a previous, known-good version after a bad deploy.

Runner — the machine (or VM) that executes a workflow's steps. GitHub Actions provides hosted runners; you can also self-host.

SSH — Secure Shell. The protocol for logging into a remote Linux server over the network.

Secret — a piece of sensitive data (token, password, key) stored encrypted by the CI system, injected into runs, masked in logs.

Workflow — a YAML file in `.github/workflows/` describing what GitHub Actions should do, when, and on what.

`workflow_run` — a trigger that fires when another named workflow completes.

Appendix B — Useful commands cheat sheet

```
# Local Docker
docker compose up -d --build      # build + start everything in background
docker compose down -v            # stop + remove volumes (wipe DB)
docker compose logs -f backend    # follow logs of one service
docker ps                         # list running containers
docker images                     # list local images

# Docker Hub
docker login                      # log in interactively
docker push <user>/<image>:tag    # push manually

# SSH to EC2
ssh -i ~/.ssh/key.pem ubuntu@<ip> # log in
scp -i ~/.ssh/key.pem file ubuntu@<ip>:~/   # copy a file to EC2

# Git
git push origin main              # trigger your pipeline
git log --oneline -10             # last 10 commits
git revert HEAD                   # safely undo the last commit (creates a new
        revert commit)

# GitHub CLI
gh run list                       # recent workflow runs
gh run watch <id>                 # watch a run live
gh run rerun <id>                 # re-run a failed run
gh secret list                    # list secret names (values are write-only)
gh secret set NAME --body "value" # create or update a secret
```

Appendix C — Where the code lives

Everything in this guide lives at:

> **github.com/LondheShubham153/github-actions-kubernetes-masterclass**

Fork it, build it, break it, fix it. That is how this becomes yours.